SELENIUM WEB DRIVER

---

1. What is Selenium WebDriver

Selenium WebDriver is a **browser automation framework** that:

- Controls real browsers
- Works at **OS level**
- Supports programming languages (Python, Java, C#, etc.)
- Interacts with web elements like a real user

Unlike Selenium IDE:

- Requires coding
- More powerful
- Used in real-world automation

---

3. Supported Browsers

- Chrome → ChromeDriver
- Firefox → GeckoDriver
- Edge → EdgeDriver
- Safari → SafariDriver

---

4. WebDriver Setup (Python)

Import Required Modules

```
from selenium import webdriver
from selenium.webdriver.common.keys import Keys
```

Launch Browser

```
driver = webdriver.Chrome("./chromedriver")
```

Creates a browser session.

Open URL

driver.get("https://www.python.org")

Equivalent to typing URL in browser.

Close Browser

```
driver.close()     # closes current window
driver.quit()      # closes all windows and ends session
```

---

5. WebDriver Properties & Browser Control

Get Page Title

driver.title

Navigation

```
driver.back()
driver.forward()
driver.refresh()
```

---

6. Locating Web Elements (CORE OF WEBDRIVER)

WebDriver **cannot interact** with elements without locators.

6.1 Types of Locators (ALL)

1. ID Locator (Best & Fastest)

HTML:

<input id="email">

Code:

driver.find_element_by_id("email")

Notes:

- Unique
- Most stable
- First preference

---

2. Name Locator

HTML:

<input name="username">

Code:

driver.find_element_by_name("username")

Notes:

- Not always unique
- First matching element returned

---

3. Class Name Locator

HTML:

<input class="inputtext">

Code:

driver.find_element_by_class_name("inputtext")

Notes:

- Only one class allowed
- Avoid if class is common

---

4. Tag Name Locator

HTML:

<a>Home

Code:

driver.find_elements_by_tag_name("a")

Notes:

- Returns multiple elements
- Mostly used for counting

---

5. Link Text Locator

HTML:

<a>REGISTER

Code:

driver.find_element_by_link_text("REGISTER")

Notes:

- Only for links
- Exact match

---

6. Partial Link Text Locator

driver.find_element_by_partial_link_text("Reg")

Notes:

- Less stable
- Used when full text changes

---

7. CSS Selector Locator (Preferred over XPath)(Most powerful)

Tag + ID
driver.find_element_by_css_selector("input#email")

Tag + Class
driver.find_element_by_css_selector("input.inputtext")

Tag + Attribute
driver.find_element_by_css_selector("input[name='email']")

Tag + Class + Attribute
driver.find_element_by_css_selector("input.inputtext[name='email']")

Notes:

- Faster than XPath
- Powerful and stable

---

8. XPath Locator (Most Flexible)

Relative XPath
driver.find_element_by_xpath("//input[@id='email']")

Using text()
driver.find_element_by_xpath("//button[text()='Login']")

Using contains()
driver.find_element_by_xpath("//a[contains(text(),'Register')]")

Notes:

- Use when no ID or name
- Avoid absolute XPath

---

7. Interacting with Web Elements

Click Element

element.click()

Enter Text

element.send_keys("hello")

Clear Text Field

element.clear()

Keyboard Actions

element.send_keys(Keys.RETURN)

---

8. Handling Dropdowns – Select Class

Used **only for `<select>` elements**.

Import

from selenium.webdriver.support.ui import Select

Create Select Object

dropdown = Select(driver.find_element_by_id("cars"))

Select Options

dropdown.select_by_visible_text("Audi")
dropdown.select_by_value("bmw")
dropdown.select_by_index(1)

Deselect Options

dropdown.deselect_all()

Get Selected Option

dropdown.first_selected_option.text

---

9. Waits in Selenium WebDriver

Used to handle **dynamic loading**.

9.1 Implicit Wait

driver.implicitly_wait(10)

Notes:

- Applies globally
- Less control
- Not recommended for large projects

9.2 Explicit Wait

Import

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

Explicit Wait Pattern

```
WebDriverWait(driver, 10).until(
    EC.presence_of_element_located((By.ID, "login"))
)
```

9.3 Expected Conditions (EC)

Common EC functions:

```
EC.title_is("Home")
EC.title_contains("Python")
EC.presence_of_element_located
EC.visibility_of_element_located
EC.element_to_be_clickable
EC.presence_of_all_elements_located
EC.alert_is_present()
```

---

11. ActionChains (Advanced User Actions)

Used for:

- Hover
- Drag and drop
- Double click

Import

```
from selenium.webdriver.common.action_chains import ActionChains
```

Create Object

actions = ActionChains(driver)

Common Actions

actions.click(element).perform()
actions.double_click(element).perform()
actions.click_and_hold(element).perform()
actions.drag_and_drop(source, target).perform()
actions.move_to_element(element).perform()

---

12. Selenium WebDriver with unittest (Brief)

WebDriver is combined with test frameworks.

```
import unittest
from selenium import webdriver

class TestApp(unittest.TestCase):

    def setUp(self):
        self.driver = webdriver.Chrome()

    def test_title(self):
        self.driver.get("https://www.python.org")
        self.assertIn("Python", self.driver.title)

    def tearDown(self):
        self.driver.quit()
```
